

Actions have consequences.

Learn from experience.

Understand reinforcement learning and robotics, from delayed rewards to reliable physical actions.

🕒 15-20 min core reading · 20-30 min practice · 12 short sections

A movie assistant can recommend something immediately or ask one question first. Asking takes effort, but the next recommendation might fit much better. Reinforcement learning studies how an agent can learn such choices from the consequences of its actions, including rewards that arrive later.

WHAT YOU WILL UNDERSTAND

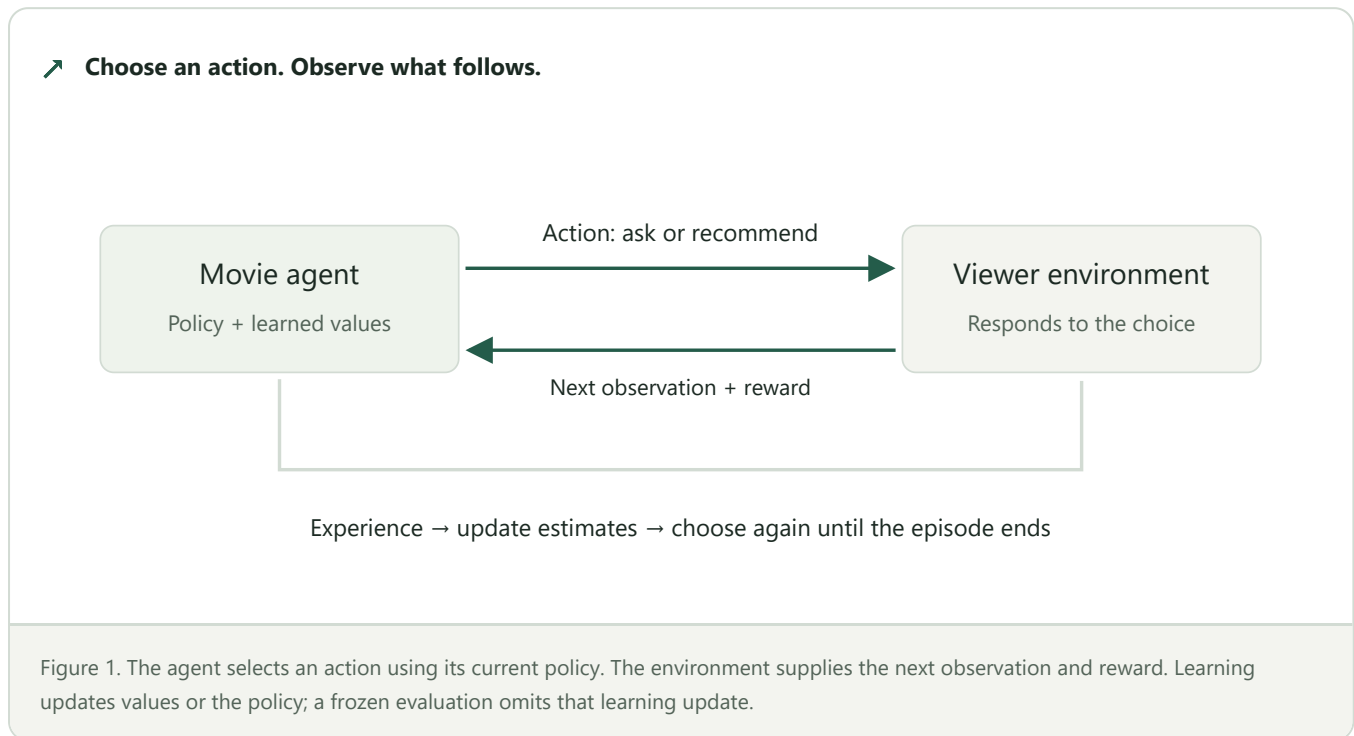
The main question: how can experience teach an agent which actions are worthwhile over time?

The practical path: define a small environment, explore actions, update their values, evaluate the learned policy, and connect the loop to physical robots.

Reading path: twelve short sections, three diagrams, a working Q-learning lab, and a Python workbook. Optional notes cover the deeper assumptions and algorithm families.

1. Learn through an action and feedback loop

In **reinforcement learning (RL)**, an **agent** chooses actions in an **environment** and learns from the resulting observations and rewards. A **policy** specifies how it chooses actions. The aim is usually to maximize expected cumulative reward, rather than predict a supplied label for each example.



Learning setup	Movie example	Feedback
Supervised learning	Predict a viewer's rating from historical examples.	A target rating for each training example.
Contextual bandit	Choose one recommendation using current context.	Reward for the chosen item; future state effects are outside the model.
Sequential RL	Decide whether to ask, recommend, or continue a session.	Rewards along a sequence influenced by earlier actions.

The agent usually does not observe what would have happened for the actions it did not choose. Its choices also influence the data it collects. This makes learning and decision-making tightly connected.

An LLM application that calls tools is not automatically an RL learner. It may execute a fixed workflow or use a previously trained model without updating any policy. Likewise, an RL agent need not use an LLM, a neural network, or a physical robot. Our lab learns four numbers in a table.

2. Describe the environment as a decision process

A **Markov decision process (MDP)** describes states, available actions, transition probabilities, and rewards. "Markov" means the current state contains the information needed to predict the next state and reward given the action; the full earlier history adds no further predictive information. See the [decision-process foundations](#).

Our fictional environment has three states: **Start**, **Preference known**, and **Terminal**. At **Start**, recommend a familiar movie or ask a question. After asking, recommend a generic or matched movie. Recommending ends the session. An **episode** is one complete session; this environment naturally ends after at most two actions.

A **reward** is one feedback value. A **return** combines rewards over time. With rewards $r_{t+1}, r_{t+2}, \dots$, a discounted return is:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

The **discount factor** γ controls how much future rewards count. At $\gamma = 0$, only the next reward matters. At $\gamma = 0.9$, the next step's reward has 90% of its undiscounted weight. A finite episode can also use $\gamma = 1$; continuing tasks often use a discount below 1 or a different objective such as average reward.

▼ Go deeper: an observation is not always a complete state

A viewer's true mood is hidden; a trailer click is only an observation. A camera image also does not reveal every object's velocity or everything behind an obstacle. A **partially observable MDP (POMDP)** distinguishes the hidden state from observations. A history, recurrent memory, or a **belief state** representing uncertainty can help an agent choose actions.

Our lab assumes "preference known" is sufficient and that every simulated viewer behaves identically. That makes the tiny state representation Markov by construction; a real recommendation system would need a richer model. For a finite-horizon task, remaining time may also belong in the state. Do not assume that a database row or the latest message automatically contains everything needed for a sound decision model.

3. Rewards determine what the agent tries to achieve

Use the following **authored satisfaction rewards**: familiar recommendation **+2**, asking **-1**, generic recommendation after asking **+1**, and matched recommendation **+6**. These numbers are teaching units, not measured ratings.

➤ **A small cost now can unlock a better outcome.**

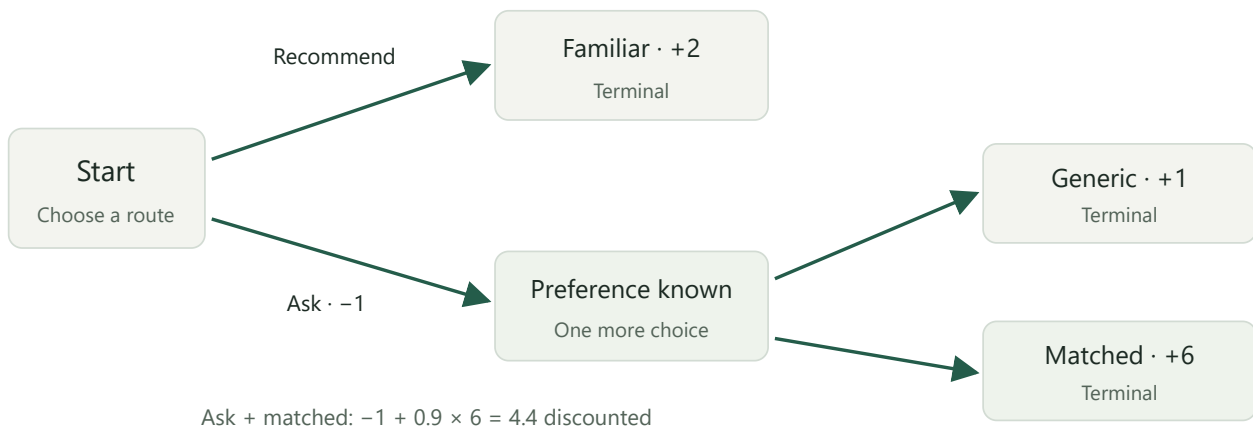


Figure 2. Satisfaction rewards in the authored movie environment. Familiar ends immediately with +2. Asking costs -1 and leads to another decision. A matched recommendation gives +6; a generic one gives +1. Every recommendation ends the episode.

Asking and then matching yields an undiscounted return of $-1 + 6 = 5$. At $\gamma = 0.9$, its discounted return is $-1 + 0.9(6) = 4.4$, beating the familiar movie's immediate 2. At $\gamma = 0$, asking is worth only -1 from Start, so the familiar recommendation wins. Choosing the immediate option in that case is correct for the specified objective.

Credit assignment asks which earlier actions contributed to a later result. Asking was initially costly, but it enabled the useful recommendation. A learner must connect those steps to discover its value.

A reward is a specification, not a direct reading of human wellbeing. The lab's alternative **click proxy** gives familiar 4, asking -1, generic 3, and matched 1. A policy optimizing this proxy chooses familiar, earning 4 proxy units but only 2 on the separate satisfaction scale. A higher number on a different reward scale is not an improvement in the original goal. [Concrete Problems in AI Safety](#) discusses problems including reward hacking and unsafe exploration.

▼ **Go deeper: sparse rewards and shaping**

A **sparse reward** appears only occasionally, such as after a whole movie session. Dense feedback can make learning easier, but intermediate rewards may encourage shortcuts. Rewarding every question could teach an assistant to keep asking instead of helping the viewer choose. **Reward shaping** adds learning signals; arbitrary shaping can change which policy is optimal. Carefully designed potential-based shaping can preserve optimal policies under its assumptions.

Keep external constraints separate from a reward tradeoff: a large engagement reward should not authorize unwanted purchases or unsafe robot motion. Inspect trajectories, not just totals, and measure the outcome you actually care about. The click-proxy experiment is deliberately simplified; real reward misspecification may be subtle and difficult to detect.

4. Balance exploration and exploitation

Exploitation chooses what currently looks best. **Exploration** tries alternatives to learn their consequences. Our **epsilon-greedy** policy selects a random valid action with probability ϵ ; otherwise it selects an action with the highest current Q value. The random branch can select that same best action, so 30% exploration does not mean 30% of actions differ from the greedy choice. See [exploration and exploitation](#).

Initially all Q values are zero. Our tie rule prefers the familiar recommendation at Start. Its positive reward increases that estimate. With $\epsilon = 0$, the agent keeps choosing familiar and never learns what asking could unlock. A zero estimate for an untried action means “unlearned here,” not “proved worthless.”

In the default 200-episode run, $\epsilon = 0.3$ eventually discovers the matched route. With only 60 episodes and the same seed, the matched action is never tried. Exploration creates an opportunity to learn; it does not guarantee good coverage in a short run.

▼ Go deeper: coverage, seeds, and convergence

Longer action sequences can require several exploratory choices before any useful reward appears. Alternatives include optimistic initial values, uncertainty-based action selection, and decaying exploration. Each makes assumptions; random actions alone can be inefficient or unsafe.

Classical tabular Q-learning convergence results require conditions such as a suitable stationary finite MDP, sufficient repeated visits to state-action pairs, and appropriate diminishing learning rates. Our fixed learning rate and finite run are for inspection, not a general convergence guarantee. The environment is deterministic, but a seeded generator makes exploration reproducible. Different seeds can produce very different early learning. In real experiments, compare multiple seeds and report variability rather than presenting one attractive curve as universal evidence.

5. Learn action values with Q-learning

A **state value** $V^\pi(s)$ is expected return from state s under policy π . An **action value** $Q^\pi(s, a)$ starts with action a and then follows that policy. The optimal action value $Q^*(s, a)$ assumes the best subsequent policy. A **Bellman relation** connects an action’s value to its immediate reward plus the value of what follows.

Q-learning estimates Q^* using observed transitions. Its **temporal-difference (TD) target** combines a received reward with an estimate for the next state:

$$y = r + \gamma \max_{a'} Q(s', a'), \quad Q(s, a) \leftarrow Q(s, a) + \alpha [y - Q(s, a)].$$

The **learning rate** α controls the update size; $y - Q(s, a)$ is the TD error. Using an existing estimate inside a target is called **bootstrapping**. The next-state term is **zero on true termination**. See [learning from experiences](#).

Our own worked update makes this concrete. Suppose the current value of asking is 0, asking gives -1 , and the best next-state estimate is 6. With $\gamma = 0.9$, the target is 4.4. With $\alpha = 0.5$, the updated value becomes **2.2**. If the next-state estimate were still zero, the same experience would initially push asking to **-0.5**. Later experience changes the target and corrects that estimate.

For a matched recommendation ending the episode, the target is simply 6. There is no next decision to value. Do not confuse a true terminal state with a training run interrupted by an external time limit: that interruption can still require bootstrapping. [Gymnasium's time-limit guide](#) explains the distinction between termination and truncation.

6. Place Q-learning among the broader methods

Tabular Q-learning stores a value for each state-action pair. Large or continuous spaces need generalization.

Deep RL uses neural networks for values, policies, or environment models; it does not remove the difficulty of selecting rewards and collecting useful experience. See [RL with generalization](#).

Family	What it learns or uses	Main idea
Value-based methods, including DQN	Action-value estimates.	Choose actions using learned values; DQN uses a neural network, replay, and a target network.
Policy gradients	A parameterized policy directly.	Adjust action probabilities to increase expected return.
Actor-critic	A policy and a value estimator.	The actor chooses; the critic supplies a learning signal.
Model-based RL	A transition/reward model, often learned.	Predict consequences and plan with that model.
Imitation learning	Behavior from demonstrations.	Copy or infer useful behavior from an expert; rewards need not be supplied.

Our learner is **model-free**: it updates from sampled transitions without using a dynamics model to plan. The code still needs an environment to generate those transitions. In **model-based RL**, a model can improve data efficiency, but mistakes in predicted dynamics can mislead planning. See the [model-based formulation](#).

▼ Go deeper: on-policy, off-policy, offline, and PPO

On-policy methods learn about the policy generating the experience. SARSA uses the next action actually selected in its TD target. **Off-policy** methods can learn about a different target policy. Q-learning uses the maximum next-state Q value even if exploration will choose something else. These distinctions concern policies, not whether the computer is connected to the internet. See [on-policy learning](#).

Online RL collects new interaction data while learning. **Offline RL** learns from a fixed logged dataset and must handle actions poorly covered by that data; being off-policy alone does not solve that problem. Replay buffers can reuse experience, but distribution shift and unstable value estimates still matter.

PPO is a policy optimization method often used with an actor-critic design; its clipped objective discourages excessively large policy updates. It does not guarantee safe behavior. See the [PPO paper](#). Chapter 28's RLHF connection uses feedback-derived rewards to train behavior; it does not mean an assistant learns a new policy from every live chat. **Behavior cloning**, by contrast, can train on demonstrated actions with a supervised objective.

7. Evaluate the learned policy separately from training

During training, rewards reflect exploratory actions and an evolving policy. During evaluation, freeze learning and use the intended deployment policy. The lab evaluates greedily with exploration off; a deployed stochastic policy would instead need evaluation with its intended randomness. See the distinction between learning-time reward and final policy quality in [RL evaluation](#).

The default run's **mean training return is 3.175**. Its final greedy policy gets **5 undiscounted** or **4.4 discounted**. These are different measurements, not contradictory answers. The lab also compares against the analytically best discounted return in this tiny environment; the learner never receives that answer during training.

For a real movie system, evaluate multiple seeds, user groups, changing tastes, and sessions absent from tuning. Compare with a simple fixed or supervised policy. Track satisfaction, completion, unwanted repetition, and constraint violations alongside the optimized reward. More interaction data has a cost; **sample efficiency** measures how much experience is needed to learn useful behavior.

Logged data only reveals outcomes for actions that were taken. Offline evaluation requires assumptions about coverage and the logging policy; a naive average over historical clicks does not establish the value of a different policy. A passing deterministic simulation establishes behavior in that model. It does not establish performance with people or physical hardware.

8. Connect the loop to robotics

A robot acts in a physical environment through **actuators**, such as motors, and receives measurements from **sensors**, such as cameras, encoders, or force sensors. Imagine a cinema robot carrying a selected movie disc to a shelf: choosing the item and reliably moving an arm are different levels of the task.

➤ **A physical action needs a closed feedback loop.**

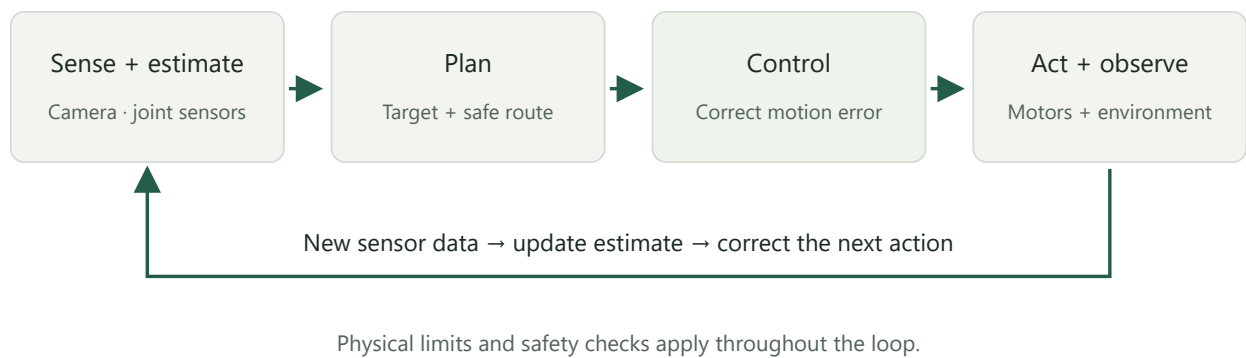


Figure 3. A cinema robot estimates what is happening, plans a movement, controls its motors, and measures the outcome. The next sensor reading closes the loop. Learning can support components of this system; it does not replace every component.

Component	Cinema-robot example
Perception	Detect the disc case and nearby obstacles in camera data.
State estimation	Estimate position and motion from noisy measurements.
Task and motion planning	Choose a pickup sequence and a collision-free route.
Feedback control	Adjust motor commands as the robot deviates from the desired motion.
Actuation and sensing	Apply forces, observe the result, and repeat the loop.

Kinematics relates joint positions to geometry and motion. **Dynamics** relates forces and torques to how motion changes. A desired hand position does not directly tell a motor which torque will safely achieve it. A learned policy may choose high-level actions or participate in control; many robots also use established planning and control methods without RL. MIT’s [Underactuated Robotics](#) develops the dynamics-and-control perspective, while its [robot simulation notes](#) connect physical models, sensors, and controllers.

Real actions take time, sensors are noisy, contacts are difficult to model, and mistakes can damage equipment. Use simulation, validated limits, constrained workspaces, and staged hardware evaluation. A reward penalty for collisions is not a substitute for collision avoidance and independent stopping mechanisms.

▼ **Go deeper: localization, feedback control, and sim-to-real**

Localization estimates where a robot is; **mapping** represents its surroundings. **SLAM** tackles localization and mapping together. **Sensor fusion** combines measurements to improve an estimate. Feedback controllers use the difference between desired and observed behavior to correct motion; **model predictive control (MPC)** repeatedly plans over a finite horizon and replans after new observations.

Simulation makes trials cheaper, but errors in friction, mass, lighting, contact, or latency create a **sim-to-real gap**. **Domain randomization** varies simulation conditions to encourage robustness; the [original visual-transfer work](#) is one example. It does not prove transfer to every physical setting. System identification, calibration, realistic disturbances, and cautious real-world testing remain important. MIT's [robotics notes](#) provide paths into estimation, control, and learning. The movie Q-table in this chapter is a conceptual bridge, not a robot controller.

9. Run the Python reinforcement workbook

Extract the ZIP and open a terminal in its folder. Use Python 3.10 or later; no packages, model calls, or credentials are needed.

```
python reinforcement.py
python reinforcement.py --epsilon 0
python reinforcement.py --gamma 0
python reinforcement.py --objective clicks
python reinforcement.py --episodes 60
python -m unittest -v test_reinforcement.py
```

File	Purpose
<code>data.json</code>	State transitions and both authored reward definitions.
<code>reinforcement.py</code>	Seeded exploration, Q updates, frozen evaluation, and reports.
<code>test_reinforcement.py</code>	Hand calculations, terminal handling, seeds, and all settings.
<code>README.md</code>	Commands, expected results, and modeling assumptions.

The default greedy path is `ask → match`, with `total: 5`, `discounted: 4.4`, and `gap: 0`. Inspect `rows` for learned Q values and visit counts, `updates` for the first twelve TD updates, and `history` for each episode. The curve groups training returns into blocks of 20 episodes.

Every run resets Q to zero. Training and action selection use full precision; the report rounds to six decimals. The seeded generator controls only exploration: rewards and transitions are deterministic. Evaluation reuses the same toy environment and therefore checks the learned policy, not generalization to unseen users.

10. Experiment with the reinforcement learning lab

Ask now. Recommend better later.

Worked runs: 200 episodes, seed 7, $\alpha = 0.5$. The default uses satisfaction, $\epsilon = 0.3$, and $\gamma = 0.9$. Evaluation turns exploration off.

Change from default	Greedy route	Discounted objective	Raw satisfaction
None	Ask $\rightarrow$ matched	4.4	5
Exploration $\epsilon = 0$	Familiar	2	2
Discount $\gamma = 0$	Familiar	2	2
Click-proxy reward	Familiar	4	2

Default mean training return is 3.175; final greedy raw return is 5. Different rewards define different objectives. At only 60 default episodes, matched has zero visits and the policy still chooses familiar.

Exercise 1: discover the delayed reward

Compare the default run with exploration set to zero. Then reset and reduce training to 60 episodes. Inspect the matched action's visit count.

▼ Reveal the explanation

Default training finds ask $\rightarrow$ match, returning 5 raw and 4.4 discounted. With zero exploration it stays with familiar, returning 2. At 60 episodes with seed 7, match has zero visits. Its Q value of zero reflects missing experience, not a measured bad outcome.

Exercise 2: change the objective without blaming the learner

Reset, then set the discount to zero. Reset again, then switch the reward to the click proxy. Why does familiar win in both cases?

▼ Reveal the explanation

At discount zero, only the next reward counts: familiar earns 2 while asking earns -1 . Under the click proxy, familiar earns 4 and is optimal for that proxy, but satisfaction remains 2. The algorithm follows the supplied objective; choosing that objective is our responsibility.

Exercise 3: calculate a TD update

Asking has Q value 0, reward -1 , and next-state maximum Q of 6. Use a discount of 0.9 and learning rate of 0.5. What changes if the next state is terminal?

▼ Reveal the explanation

The nonterminal target is $-1 + 0.9 \times 6 = 4.4$, so the new Q value is 2.2. If that transition truly terminates, omit future value: target -1 , new Q -0.5 . The lab's asking action is nonterminal; the terminal version is a separate arithmetic check.

11. Check your understanding

01 Asking earns -1 now and enables a matched recommendation worth $+6$ next. With discount 0.9 , what is the return from asking and then matching?

- A. 4.4 discounted; 5 undiscounted.
- B. 5 discounted; 4.4 undiscounted.
- C. -1 , regardless of the discount.

Answer: A. The next reward is discounted once: $-1 + 0.9 \times 6 = 4.4$. Adding without discount gives 5 .

02 An action has $Q = 0$ and zero visits. What does that tell you in this lab?

- A. The action is known to have zero reward.
- B. Its value is still the initialization, without direct experience.
- C. It is forbidden by the environment.

Answer: B. Unvisited does not mean bad. Inspect coverage before interpreting learned estimates.

03 How should a Q-learning target handle a transition to a true terminal state?

- A. Always add the largest Q value in the table.
- B. Pretend the next episode continues the same return.
- C. Use the reward alone; there is no next-state value to bootstrap.

Answer: C. A true terminal transition ends the return. An external truncation can require different handling.

04 The policy achieves a higher click-proxy score but lower satisfaction. What is the central issue?

- A. All reinforcement learning algorithms are broken.
- B. The optimized reward may not represent the desired outcome.
- C. More training must make both metrics identical.

Answer: B. A learner can optimize its supplied objective successfully while missing the human goal. Validate the reward and evaluate other outcomes.

05 A robot policy succeeds in simulation. What follows?

- A. It is safe to run at full speed around people.
- B. A collision penalty replaces physical safety measures.
- C. Check model mismatch, sensing, control limits, and staged hardware performance.

Answer: C. Simulation is evidence about a model. Physical deployment adds dynamics, noise, timing, and safety constraints that need independent validation.

12. Revise and connect the ideas

Explain the complete chain: **the environment produces experience; exploration determines what is tried; rewards define the objective; value updates propagate credit; the policy selects actions; evaluation checks the outcome.** Then name what changes when the environment becomes a physical robot.

▼ Quick-reference glossary

Term	Meaning
Agent / environment	Decision-maker / system it interacts with.
Policy	A rule or distribution for choosing actions from available information.
MDP / POMDP	A decision process with a Markov state / with partial observations.
Episode / terminal state	One interaction sequence / a state where that sequence ends.
Reward / return	One feedback value / an accumulation of rewards.
Discount / learning rate	Weight on future reward / size of a learning update.
Exploration / exploitation	Gather experience / use current estimates.
Q value / TD error	Estimated action return / target minus current estimate.
Bootstrapping	Using an existing estimate in another learning target.
Actor / critic	Policy component / value-estimation component.
Model-free / model-based	Learn without a planning model / use a model of consequences.
Kinematics / dynamics	Geometry and motion relationships / forces and resulting motion.
Feedback control / sim-to-real	Correct motion using observations / transfer from simulation to hardware.

The linked textbook sections, research papers, Gymnasium documentation, and MIT robotics notes provide the formal foundations. The movie environment, reward scores, diagrams, and workbook are original teaching examples. RL is useful when sequential consequences matter and suitable feedback can be collected; it is not required for every recommender or robot.

Next: Chapter 39 - Vision, recommendations, and time-series applications. We will connect the techniques already learned to practical application designs and their evaluation.